

TLS Fingerprinting with JA3, JA3S and JA4

1. Introduction

This project builds a tool that identifies the software behind a TLS connection by looking at the structure of its handshake. It reads a packet capture, extracts the ClientHello and ServerHello, computes JA3, JA3S and JA4 fingerprints, and matches them against a reference database measured from five different clients. Only the plaintext preamble of the handshake is read.

A second component compares a client's stated identity against its measured fingerprint, which is how the technique is applied in bot detection products.

`docs/BACKGROUND.md` covers the protocol details and the construction of each fingerprint. This report assumes that material and concentrates on what was built, what was measured, and what the measurements mean.

2. Motivation

Most web traffic is encrypted. A monitoring system on the network can no longer read URLs, headers or payloads. It can still read the handshake, since the first two messages have to travel in plaintext.

Those messages are not uniform. Each TLS library ships its own defaults for which cipher suites to offer and in what order, which extensions to include, and which curve groups to accept, and applications layer their own configuration on top. The opening message of a connection therefore carries enough structure to tell a browser apart from a scripting library.

That gives a defender something to work with when the traffic itself is opaque. It is also approximate and evadable, and both properties show up in the results below.

3. What the tool does

Given a `.pcap` file:

1. Packets are grouped into TCP flows by address and port pair.
2. Each direction of each flow is reassembled into one byte stream, ordered by sequence number.
3. TLS records are walked and handshake messages of type `0x01` (ClientHello) and `0x02` (ServerHello) are extracted, including messages split across several records.
4. Those messages are parsed field by field against RFC 5246 and RFC 8446.
5. JA3 and JA4 are computed from the ClientHello, JA3S from the ServerHello.
6. Each value is looked up in `data/fingerprint_db.json` and reported.

4. Architecture

```
.pcap file  
-> scapy.rdpcap
```

```
-> group into TCP flows, reassemble each direction      (analyzer.py)
-> walk TLS records, extract the two hellos              (parser.py)
-> build JA3, JA3S, JA4                                (ja3.py, ja3s.py, ja4.py)
-> look up in the reference database                     (database.py)
-> format and print                                     (report.py, cli.py)
```

The modules do not know about each other beyond the data they pass along. `analyzer.py` is the only one that handles packets. The three fingerprint modules are pure functions over parsed structures, which is what makes them testable against published values.

5. Implementation notes

5.1 Parsing

`parser.py` reads raw TLS bytes against the RFC layout instead of using Scapy's TLS layer. Two reasons. Every offset the parser reads can be pointed at a line in the specification, which matters when the output is a hash and a wrong offset produces a plausible-looking wrong answer rather than an error. And reading two messages does not justify depending on a full TLS stack. The original Salesforce implementation takes the same approach.

The parser stops rather than guessing when a record is truncated, and returns `None` rather than raising when a stream contains no hello. Feeding it three thousand mutated and truncated ClientHellos produced no uncaught exceptions.

5.2 Reassembly

Payloads in each direction are ordered by sequence number and deduplicated. This handles segmentation, out-of-order delivery and simple retransmission, all of which were tested. Overlapping segments with conflicting content and sequence number wraparound are not handled. Neither appears in the captures used here, and both would matter in a production capture.

One known gap: two successive connections that reuse the same four-tuple in one capture are merged into a single flow, and only the first ClientHello is reported.

5.3 Database

`database.py` holds a validated dataclass backed by a JSON file. A lookup returns one of three states.

`known` means one entry carries that hash. `possible` means several entries with different names share it, and the tool lists all of them. `unknown` means the hash is not on file.

`possible` exists because collisions are a property of the technique rather than an error in the implementation. Reporting a single name in that case would be a guess presented as a result.

5.4 Capture

`capture_proxy.py` and `pcap_write.py` produce the experiment captures and are not part of the analysis path. Section 10 describes what they do and what follows from it.

6. Method

Five programs were pointed at `example.com`, a domain IANA reserves for documentation and testing:

1. curl 8.7.1, the macOS system build, on SecureTransport over LibreSSL 3.3.6
2. openssl s_client 3.6.2, installed through Homebrew
3. Python 3.14.6 using `ssl.create_default_context()`
4. Google Chrome 151 headless, on BoringSSL
5. A ClientHello assembled by hand over a raw socket, with no TLS library

The set was chosen to separate three variables. curl and Chrome are different libraries entirely. openssl s_client and Python share a library and differ only in configuration, which tests whether the fingerprint tracks the library or the caller. The hand-built hello has no library at all: it was written byte by byte for this project, offers legacy TLS 1.2, and omits supported_versions and key_share on purpose.

Two further experiments came out of observations made while testing rather than from the plan:

6. Chrome was captured a second time, to test whether one install produces a stable fingerprint across runs.
7. A script sending a real Python ssl handshake under a spoofed Chrome User-Agent was captured, to test whether the handshake contradicts the claim.

7. Results

7.1 Five clients

Client	TLS library	JA3	JA3S
curl 8.7.1	SecureTransport over LibreSSL	375c6162a49 2...ce8424	d75f9129bb5 d...e081bcb 2
openssl s_client 3.6.2	OpenSSL 3.6.2	0b85eb0d498 1...f0ac5f	907bf3ecf1 c...37b43de 8
Python 3.14.6 ssl	OpenSSL 3.6.2	f21f8e6cf70 d...ef401c	907bf3ecf1 c...37b43de 8
Chrome 151 headless	BoringSSL	81a2542af84 4...f2a626	eb1d94daa7e 0...fb6e705 4
hand-built ClientHello	none	c53113116bb 0...6fedc9	ba02d4299a6 e...7631993

All five JA3 hashes differ, and the tool identifies each capture correctly from its handshake with no other information.

Rows two and three answer the question the client set was chosen to answer. They link against the same OpenSSL 3.6.2 build and produce different JA3 hashes, because

`ssl.create_default_context()` curates a shorter and differently ordered cipher list than the `openssl`

command line default. The fingerprint tracks the configuration a program presents, not the library it loads.

The same two rows also produced an identical JA3S. Their offers overlapped enough that Cloudflare made the same choice both times. Section 10 covers what that implies for reading a JA3S.

7.2 One Chrome install, two runs

Run	JA3	JA4
First	81a2542af84 42fcd7802f1 78d9f2a626	t13d1516h2_ 8daaf615277 1_806a8c22f dea
Second	a00e551d2f4 af85ede1156 537ebf095a	t13d1517h2_ 8daaf615277 1_541cd5a3d 78e

Two runs of the same binary, minutes apart, gave two unrelated JA3 hashes. Chrome randomises the order of its ClientHello extensions on every connection, and JA3 hashes extensions in the order they were sent. This is not a bug in either the browser or the tool. Chrome does it to make JA3 stop working, and the result here is what that looks like.

The JA4 cipher segment, 8daaf6152771, is identical across both runs. That segment is the hash of the cipher list after sorting, so a reordering cannot change it. This is the property JA4 was designed for, measured rather than assumed.

The rest of the JA4 string did change. The extension count went from 16 to 17, because the second run offered one extension the first did not. That is a difference in what the client sent, not a reordering, and no order-insensitive scheme should hide it. The comparison is only possible because JA4 splits the handshake into a readable count, a cipher hash and an extension hash. JA3 reduces all of it to one number, so a reordering and a genuine change are indistinguishable in the output.

7.3 Identity claim checking

An HTTP User-Agent is a string the client picks. A script can claim to be Chrome in one line of code, and a server has no way to check it from the header alone. The ClientHello was already sent before that header existed, produced by whichever TLS library the script actually links against.

`experiments/bot_client.py` sends a Chrome User-Agent over a plain Python `ssl` handshake. The tool reports a mismatch and names Python's `ssl` module as the source of the fingerprint. Repeating the request five times over five separate connections produced five flags: each connection carries its own handshake, so there is no running average to dilute and no per-source threshold to stay under. Distributing the same script across many addresses would not help either, since the address is not part of the comparison.

The limits of this are worth stating alongside the result. The detection works because the script uses an unmodified Python `ssl` handshake. `curl-impersonate` and `uTLS` reproduce Chrome's ClientHello byte for byte, and a real browser under automation sends a real browser's handshake. Both pass this check.

What it catches is a fake label on an off-the-shelf script, which is the cheap attack and also the common one.

7.4 Verification

The suite contains 61 tests.

JA3 and JA3S string construction is checked against values worked out by hand from the RFC field layout, so the code is compared against the specification rather than against its own output. JA4 is checked against the worked examples published in the FoxIO specification, which is ground truth from outside the project. The remaining tests cover parser edge cases (a truncated record, a ClientHello offering only GREASE ciphers, a handshake split across two TLS records, a malformed extension length), database lookup behaviour, the identity-claim comparison, one end-to-end run from `.pcap` to printed report, and a check that every database entry still reproduces from the capture it was measured from.

That last test was added after a stale hash was found in the documentation. It had drifted out of agreement with the captures and nothing in the suite could notice.

Separately, every fingerprint was compared against Wireshark, which implements JA3, JA3S and JA4 independently:

```
tshark -r pcaps/curl.pcap -Y "tls.handshake.type==1" \
-T fields -e tls.handshake.ja3 -e tls.handshake.ja4
```

The values agree on every capture in `pcaps/`, for all three fingerprint types.

8. A conformance defect found by cross-checking

The Wireshark comparison disagreed on one capture.

This implementation had been removing GREASE values from the `signature_algorithms` list before hashing it into JA4's third segment. The JA4 specification lists GREASE removal for the cipher list and the extension list and says nothing about signature algorithms, and Wireshark's dissector keeps them.

The disagreement only appears when a client puts GREASE in that list. Modern Chrome does.

`pcaps/chrome_live.pcap` carries `0x4a4a` as its first signature algorithm, and it is the only capture in the set that does:

```
tshark:          t13d1517h2_8daaf6152771_541cd5a3d78e
before the fix:  t13d1517h2_8daaf6152771_cb7bf5808d99
```

The filter was removed and a regression test added that passes a signature algorithm list containing `0x4a4a` through the extension hash and asserts the value Wireshark reports. Every capture now agrees with Wireshark on all three fingerprint types.

The defect is worth reporting because of how it was found. It survived 56 passing tests, including tests written against the JA4 specification's own examples, because none of those examples contains a GREASE signature algorithm. A test suite checks that the code does what its author believed the specification said. Comparing against a second implementation checks the belief.

9. Security applications

Command and control detection. Malware families reuse their TLS stack across samples and campaigns, and a hand-rolled or unusual stack stands out against ordinary browser and operating system traffic even when the traffic cannot be read. The hand-built client in this project is that shape: four cipher suites, four extensions, no `supported_versions`, nothing else on the network looks like it.

Anomaly detection. A JA3 never seen before from a given host is useful without knowing what produced it.

Asset inventory. Finding which TLS library versions are active on a network without installing anything on the endpoints, and noticing when one changes.

Pairing client and server. A JA3 read together with the JA3S it produced narrows identification further than either alone, and is the standard way of tracking a specific malware family talking to a specific controller.

Identity claim verification. Section 7.3. This is the application the project implements end to end, and the one behind commercial bot mitigation.

10. Limitations

An identical hash does not mean identical software. Two unrelated programs on the same library with the same configuration produce the same JA3. The database reports this as `possible` and lists every candidate.

A JA3S depends on the client. Two clients here received the same JA3S from the same server because their offers overlapped enough for the server to choose identically. A JA3S describes a client and server pair.

GREASE and extension-order randomisation exist to weaken this technique. GREASE is handled by stripping it where the specifications say to. Chrome's per-connection reordering cannot be undone by any fingerprint that hashes order, and section 7.2 shows it defeating JA3. JA4 survives reordering and does not survive a real change in what the client offers.

Evasion is straightforward. Every byte of a ClientHello is chosen by the client, so a program can send another program's exact cipher and extension list. Tools that do this are public and maintained. A clean fingerprint is therefore not evidence of anything.

Fingerprints drift. Library and operating system updates move a program to a new JA3, so every database entry records the platform and date it was measured on.

Capture method. The files in `pcaps/` were not taken from a network interface. Live capture needs root, which was not available in the environment this was built in, so a local TCP relay was used: the client connects to the relay, the relay forwards every byte to the real server without modification while keeping a copy, and that copy is written out as a `.pcap`. The TLS bytes are what the client sent and the server returned, and certificate validation succeeded end to end on every capture. The framing around them is synthetic, so each file holds one packet per direction rather than a full TCP conversation. Every database entry records this in its `source` field. The analysis pipeline has no such limitation and reads ordinary captures, including segmented handshakes, out-of-order delivery and concurrent flows.

11. Future work

Completed during the project, originally listed as a stretch goal: JA4 for the client side, and the identity-claim detector.

Not attempted:

JA4S, JA4H, JA4X and JA4SSH, the rest of the JA4 family. JA4S in particular would complete the pairing described in section 9. JARM, which fingerprints servers actively by sending crafted hellos rather than observing traffic, is a different technique with the same purpose.

A larger reference database. Five clients on one operating system demonstrates the technique and is not enough to deploy it. More tools, more platforms, and entries taken from published reference sets with citations would all help.

Live capture from a network interface, replacing the relay described in section 10.

Capture at higher rates with eBPF or XDP, computing fingerprints in kernel space instead of copying every packet to userspace. Linux only, and unnecessary at the volumes here.

A key-value store such as Redis behind the lookup instead of a JSON file, which starts to matter once per-flow lookup latency is a consideration.

Reading the `User-Agent` from traffic rather than taking it as an argument. This would require terminating TLS and was avoided on purpose.

12. Conclusion

The tool works: a packet capture goes in, a named client comes out, using only the plaintext part of the handshake. It was validated on five clients that produce five different fingerprints, including two that share a crypto library and differ on configuration alone, and cross-checked against Wireshark on every capture.

Two of the more useful findings were not planned. The same Chrome install fingerprinted differently across two runs, which is JA3 failing in exactly the way its critics describe, and having JA4 alongside it made it possible to say which part of the handshake had moved. The Wireshark comparison found a conformance defect in this project's own JA4 code that the test suite could not have caught.

The limitations are part of the result. A fingerprint reads bytes the client chose to send, so a client that chooses differently reads as something else. It is a signal to correlate with other evidence, and treating it as an identity is where deployments of this technique go wrong.

References

Salesforce, JA3: <https://github.com/salesforce/ja3>

FoxIO, JA4: https://github.com/FoxIO-LLC/ja4/blob/main/technical_details/JA4.md

RFC 5246, The Transport Layer Security (TLS) Protocol Version 1.2

RFC 8446, The Transport Layer Security (TLS) Protocol Version 1.3

RFC 8701, Applying Generate Random Extensions And Sustain Extensibility (GREASE) to TLS
Extensibility